IEEE ICCC 2023 1570913943

Adaptive Timescale Load Balancing Routing Algorithm for LEO Satellite Network

1st Shuyang Liu
Department of Electronic Engineering
Shanghai Jiao Tong University
Shanghai, China
Email: hello_world@sjtu.edu.cn

2nd Jing Liu
Department of Electronic Engineering
Shanghai Jiao Tong University
Shanghai, China
Email: jingliu_lj@sjtu.edu.cn

3rd Bin Xia
Department of Electronic Engineering
Shanghai Jiao Tong University
Shanghai, China
Email: bxia@sjtu.edu.cn

2023 IEEE/CIC International Conference on Communications in China (ICCC) | 979-8-3503-4538-4/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICCC57788.2023.10233496

***Abstract*—The LEO satellite network demonstrates excellent communication capabilities, including uninterrupted service provision, large bandwidth, low delay, and wide geographic coverage, highlighting its important role in global connectivity. Due to the uneven distribution of global traffic, the links between developed regions are more prone to congestion, while other links are relatively idle. In order to improve the throughput of the network while ensuring good end-to-end delay performance, an adaptive timescale load balancing (ATLB) routing algorithm based on SDN is proposed in this paper. By modeling the link historical traffic through an approach based on empirical mode decomposition, ATLB classifies congestion as transient and persistent. When a transient congestion occurs, the pre-configured alternative forwarding path will work for local load balancing. And when persistent congestion occurs, ATLB will perform global load balancing on the appropriate timescale determined by the control plane adaptively. The simulation results show that ATLB has better comprehensive performance in packet drop rate, end-to-end delay and throughput compared with other algorithms.**

***Index Terms*—LEO satellite network, load balancing, SDN, historical traffic, adaptive timescale**

I. Introduction

With the increasing number of Internet-connected devices and traffic, terrestrial networks face challenges in capacity and service levels [1]. To address these shortcomings, Low Earth Orbit (LEO) satellite networks have emerged as a promising solution due to their all-weather capabilities, high bandwidth, wide coverage, and freedom from terrain constraints [2]. The application of LEO satellite networks for Internet services is becoming a future trend in communication network development. Recent advancements in the aerospace field have further accelerated the adoption of satellite networks. However, the distribution of global traffic is uneven, with developed regions experiencing higher traffic generation and interaction, leading to congestion in corresponding Inter Satellite Links (ISLs). In contrast, other ISLs remain relatively underutilized. To tackle the congestion arising from unbalanced load, it is crucial to study load balancing routing protocols in LEO satellite networks.

Researchers have explored load balancing routing protocols for LEO satellite networks. For instance, in [3], a decentralized approach called ELB achieves local load balancing by explicitly notifying neighbors of the load status. Additionally, a public queue mechanism is introduced on satellites in TLR [4], alleviating congestion by utilizing resources from other directions. Other methods, such as LCRA [5] and DBPR [6], have been proposed to optimize throughput and reduce end-to-end delay. While these methods respond quickly and have low overhead, their effective scope is limited, and cascading congestion may occur if congested links' neighbor links are also under high load. To address these challenges, some solutions based on Software-Defined Network (SDN) have been proposed. In [7], a master controller and slave controllers on Earth optimize routing paths and bypass congested links for load balancing. Similarly, [8] divides the network into clusters to minimize signaling overhead and applies load balancing within each cluster. Another SDN-based approach in [9] models the LEO satellite network as a time-varying topology and utilizes multi-attribute decision-making to dynamically calculate forwarding paths.

Despite their flexibility and adaptability, these SDN-based methods rely solely on the current network state obtained by the control plane, overlooking the historical network state. To address this limitation, we propose an adaptive timescale load balancing routing algorithm (ATLB). By incorporating empirical mode decomposition to process historical network information, we distinguish transient congestion and persistent congestion. For transient congestion, routing decisions are automatically adjusted to ensure timely load balancing in congested areas, while persistent congestion is addressed by adopting ISLs outside the congested regions for global load balancing.

The rest of this paper is organized as follows: Section II presents the satellite system model and the SDN architecture. Section III provides a detailed description of the proposed ATLB algorithm. Section IV presents simulation results and analysis, while Section V summarizes the paper's findings.

II. System Model

Considering Iridium-like constellation in this paper, there are M orbits with N satellites on each one. Each satellite around the reverse seam has two intra-plane ISLs and one inter-plane ISL, while others have two intra-plane ISLs and two inter-plane ISLs. When the satellite enters the polar region, the inter-plane ISLs will be temporarily disconnected and resumed after the satellite leaves the region.

Authorized licensed use limited to: AJOU UNIVERSITY. Downloaded on November 05,2025 at 11:11:18 UTC from IEEE Xplore. Restrictions apply.

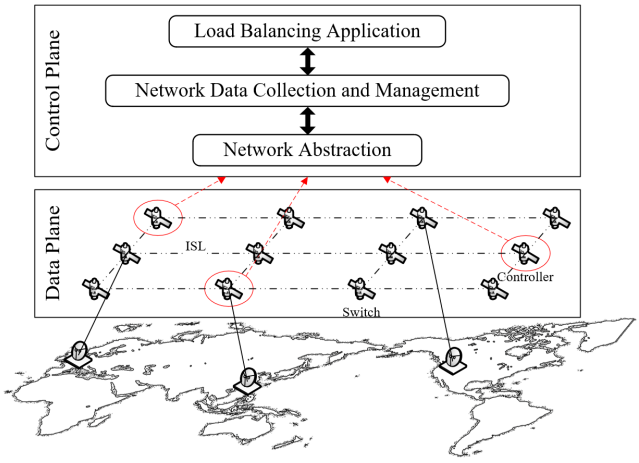


Fig. 1. The architecture of the network.

The LEO satellite network adopts SDN architecture, as shown in Fig.1, all satellites in the constellation constitute the data plane and several of them also act as controllers to form the control plane. In this software-defined satellite network, all satellite nodes are switches, and distributed controllers manage the network through them. On the data plane, switches are connected through ISLs to form a network and are responsible for forwarding packets. On the control plane, the network is abstracted into a topology, and the network data will be collected and managed. Since the relatively developed regions such as East Asia, North America and Western Europe have higher communication requirements than other regions, deploying the load balancing application on the control plane is an important measure to improve the performance of throughput and packet drop rate.

III. Adaptive Timescale Load Balancing Routing Algorithm

To achieve load balancing through the network application deployed on the control plane, based on SDN, the proposed ATLB routing algorithm will record and store the information of the traffic through each ISL. Therefore, both current and historical network states will be considered comprehensively by the control plane. The load balancing application will adopts different strategies after higher load nodes are analyzed by the control plane when load balancing is required. The ATLB will be described through three modules. The link historical traffic model based on empirical mode decomposition provides a mechanism for collecting and maintaining the link historical information. Based on it, the congestion classification mechanism provides a method to classify congestion. The third module introduces the whole process of load balancing.

A. Link Historical Traffic Model Based on Empirical Mode Decomposition

The topology of the network is abstracted as a graph $G = (V, E)$, where $V = \{S_{<1,1>}, S_{<1,2>}, \cdots, S_{<M,N>}\}$ and $E = \{l = (S_{<i_1,j_1>}, S_{<i_2,j_2>}) \mid 1 \le i_1, i_2 \le M, 1 \le j_1, j_2 \le N\}$ are the satellite set and the satellite network edge set, respectively. Link l is monitored by satellite $S_{<i_1,j_1>}$, and the link traffic information is notified to the control plane every Δt. Assume that there are n packets need to be transmitted through l on $S_{<i_1,j_1>}$ within Δt, the link traffic information is expressed as

$$B_l = [L_1, L_2, \cdots, L_n] \tag{1}$$

where L_i denotes the length of the i-th packet P_i within Δt.

The flows transmitted in network G constitutes the flow set F, the control plane maintains the global information of the network and assigns forwarding path $path_f$ for each flow f. A flow forwarding logic matrix A is formed on the control plane:

$$A = (a_{f,l})_{|F|\times|E|}, \quad a_{f,l} = \begin{cases} 0, & l \notin path_f \\ 1, & l \in path_f \end{cases} \tag{2}$$

During Δt, $S_{<i_1,j_1>}$ records the total amount of data r^l and the partial amount of data belong to f through l r_f^l :

$$r^l = \sum_{i=1}^{n} L_i \tag{3}$$

$$r_f^l = \sum_{i=1}^{n} L_i \theta_{P_i}^f \tag{4}$$

$$\theta_{P_i}^f = \begin{cases} 0, & P_i \notin f \\ 1, & P_i \in f \end{cases} \tag{5}$$

The control plane will receive a notification of l and store its traffic information r^l and r_f^l every Δt. These information are discrete time series in chronological order. Assume that discrete time series X_l and X_l^f are link historical traffic information, referred to total amount of data and partial amount of data belong to f, respectively. And the following equation holds:

$$X_l(i) = \sum_{f \in F} a_{f,l} \cdot X_l^f(i) \tag{6}$$

where $i = 1, 2, \cdots, k$, $X_l(k)$ and $X_l^f(k)$ are the latest link traffic information of l received by the control plane.

Without any prior knowledge, the signal will be decomposed into several Intrinsic Mode Functions (IMF) adaptively according to its characteristic by Empirical Mode Decomposition (EMD). Note that making no priori assumption about link historical traffic is desired and we hope to obtain a modeling method that can be adaptively adjusted according to the link historical traffic characteristics. Therefore, the EMD algorithm is used to model link historical traffic. Each intrinsic mode function needs to satisfy:

- The difference between the number of extreme points and the number of zero-crossing points of the series is not greater than 1.
- The average value of the upper and lower envelope is always 0.

Taking X^l as example, Algorithm 1 illustrates the workflow of the EMD algorithm.

Authorized licensed use limited to: AJOU UNIVERSITY. Downloaded on November 05,2025 at 11:11:18 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: EMD algorithm

Input: Series to be decomposed: X_l
Output: Intrinstic mode functions: $IMF = \{Imf_j\}$;
Residual series: R;

Initial $k = 1$, $X = X_l$, $R = X_l$, $j = 1$, $IMF = \varnothing$;
Obtain the upper and lower envelope of X: X_+ and X_-;
Compute the average value of the upper and lower envelope: $M_k = \frac{X_+ + X_-}{2}$;
Compute the intermediate series: $H_j^k = X - M_k$;
if H_j^k *satisfy intrinsic mode function conditions* **then**
  $Imf_j = H_j^k$;
  $IMF = IMF \cup \{Imf_j\}$;
else
  $X = H_j^k$;
  $k = k + 1$;
  go to line 2;
end
compute the residual series: $R = R - Imf_j$;
if R *is monotonic* **then**
  return IMF, R;
else
  $X = R$; $j = j + 1$; $k = 1$; go to line 2;
end

Then, X_l is decomposed into the sum of intrinstic mode functions and residual series:

$$X_l = \sum_{j=1}^{|IMF|} Imf_j + R \tag{7}$$

Different Imf_j correspond to different timescales, Imf_1 contains the component with the smallest timescale in X_l while $Imf_{|IMF|}$ corresponds to the largest timescale. As j increases, the corresponding timescale of intrinstic mode function becomes larger, as shown in Fig. 2.

The intrinstic mode function reflects the trend of the original sequence on the corresponding timescale, the high-order intrinstic mode function is smoother, and it represents the trend of the sequence relatively macroscopically, while the low-order is the opposite. Therefore, to characterize link historical traffic, mean square error $\mathcal{E}$ and smoothness $\mathcal{S}$ are considered, where $\mathcal{E}$ represent the mean square error of X_l and its corresponding series and $\mathcal{S}$ represent the smoothness of the corresponding series at j-th timescale:

$$\mathcal{E} = \frac{\sum_{i=1}^{k-1} \big(R(i) + Imf_j(i) - X_l(i)\big)^2}{k-1} \tag{8}$$

$$\mathcal{S} = \frac{\sum_{i=1}^{k-1} \big(R(i+1) - R(i) + Imf_j(i+1) - Imf_j(i)\big)^2}{k-1} \tag{9}$$

With $|IMF|$ intrinstic mode functions, $\bar{j}$ can be determined by:

$$\min_j \ \big(\alpha \cdot \mathcal{E} + (1-\alpha) \cdot \mathcal{S}\big) \tag{10}$$

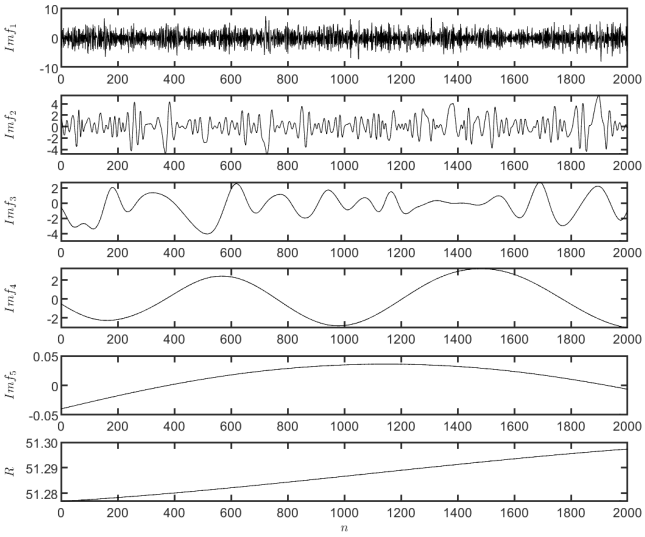


Fig. 2. Intrinstic mode functions and residual series.

where α is a constant in [0,1]. $\bar{j}$ is a variable determined adaptively according to the characteristics of the link historical traffic. Taking $Imf_{\bar{j}}$ as a boundary, X_l is decomposed into large timescale series LTS_l and small timescale series STS_l:

$$LTS_l = R + \sum_{j=\bar{j}}^{|IMF|} Imf_j \tag{11}$$

$$STS_l = \sum_{j=1}^{\bar{j}-1} Imf_j \tag{12}$$

Similarly, each flow f on link l can also be described in the same way as LTS_l^f and STS_l^f.

B. Congestion Classification Mechanism

In LEO satellite network, link congestion is first perceived by the satellite. Assume that congestion occurs on link $l = (S_{<i_1,j_1>}, S_{<i_2,j_2>})$, and the control plane will be notified by $S_{<i_1,j_1>}$. Then, the burst degree $\mathcal{D}$ of the congestion can be obtained by:

$$\mathcal{D} = \frac{\sum_{i=1}^{k} \gamma\big(STS_l(i), X_l(k) - LTS_l(k)\big)}{k} \tag{13}$$

$$\gamma(a,b) = \begin{cases} 0, & a > b \\ 1, & a \le b \end{cases} \tag{14}$$

The burst degree of congestion is defined as the probability of taking the difference between X_l and LTS_l as the lower quantile on STS_l, which represents the proportion of non-congested time to the total time on the small timescale. Under the timescale of LTS_l, a high proportion indicates a high burst degree.

Assume that the zero point number of $Imf_{\bar{j}}$ is s, then the quantization of the corresponding timescale Ts is $[\frac{k}{2s}]$, $[\cdot]$ is

Authorized licensed use limited to: AJOU UNIVERSITY. Downloaded on November 05,2025 at 11:11:18 UTC from IEEE Xplore. Restrictions apply.

rounding symbol. The excess traffic can be calculated by the control plane:

$$D_l = \sum_{i=k-Ts+1}^{k} \max\ \left(STS(i) - X_l(k) + LTS(k), 0\right) \quad (15)$$

And the excess traffic of f passing through l is:

$$D_l^f = D_l \cdot \frac{LTS_l^f(k)}{\sum_{f' \in F} a_{f',l} \cdot LTS_l^{f'}(k)} \quad (16)$$

The control plane determines whether to intervene in load balancing through the burst degree of the congestion and the excess traffic to perform the global load balancing.

C. Load Balancing Mechanism

The proposed ATLB configures the forwarding path for the new arriving flow according to the shortest path algorithm when the network is not congested, so as to ensure the performance under light load. At the same time, the control plane records the configured forwarding path and thus updates and maintains the forwarding logic matrix A. Due to the delay of the control plane and the data plane, when the congestion occurs, the control plane cannot immediately respond to the congestion. Therefore, the control plane pays attention to the links with heavy traffic, and pre-configures alternative forwarding paths for them.

When congestion occurs on link l, $S_{<i_1,j_1>}$ will forward excess traffic to the alternative path and notify the control plane at the same time. Algorithm 2 illustrates the pre-configuration of alternative path.

Algorithm 2: The pre-configuration of alternative path

initial $l = (S_{<i_1,j_1>}, S_{<i_2,j_2>})$, $P = \varnothing$,
  $neighbor$=`GetNeighbors(`$S_{<i_1,j_1>}$`)` ;
for *each flow* f *passing through link* l **do**
  for *each* $s \in neighbors$ **do**
    if $s == S_{<i_2,j_2>}$ **then**
      continue;
    else
      $path$=`Dijkstra(`s,$destination_f$`)`;
      if $\forall\ l' \in path,\ LTS_{l'} < LTS_l$ **then**
        $P = P \cup \{path\}$;
      else
        continue;
      end
    end
  end
  $path^f_{alternative}$=`FindShortest(`P`)`;
  configure $path^f_{alternative}$;
end

If the burst degree $\mathcal{D}$ of a congestion is greater than 50%, then the congestion is transient and the excess traffic will be forwarded through the alternative path. Conversely, the congestion is persistent, and the control plane performs global load balancing. The Algorithm 3 illustrates the workflow of the global load balancing.

Algorithm 3: The global load balancing.

for *each flow* f *passing through link* l **do**
  initial $G' = (V', E') \leftarrow G$;
  for *each* $l' \in E$ **do**
    if $LTS_{l'} \geq LTS_l$ **then**
      delete l' in E';
    end
  end
  $path$ = `Dijkstra(`$source_f$,$destination_f$`)`;
  configure $path$;
  $source_f$ shunts traffic to $path$ at data rate $\frac{8D_l^f}{Ts \cdot \Delta t}$ during $Ts \cdot \Delta t$
end

IV. Simulation And Performance Evaluation

A. Parameters

In this paper, an Iridium-like satellite constellation based on NS-3 is used for simulation. The constellation consists of 66 satellites, evenly distributed on 6 orbital planes with an altitude of 780km and an inclination of 90°. The simulation parameters are listed in Table I. In this scenario, the proposed ATLB routing algorithm is compared with DBPR [6], LRES [10] and the Dijkstra algorithm.

B. Simulation Results and Analysis

Fig.3 presents the Packet Drop Rate (PDR) with increasing average bitrate per flow. The results show that when the average bitrate per flow rises from 1.4 to 1.8 Mbps, both ATLB and DBPR exhibit superior load balancing performance compared to LRES and Dijkstra, with similar PDR values. This indicates that the pre-configured alternative forwarding path in ATLB and the distributed gradient forwarding mechanism in DBPR effectively balance loads under lighter conditions. As the average bitrate per flow surpasses 1.8 Mbps, DBPR's PDR becomes the lowest, while ATLB's PDR approaches DBPR, outperforming LRES and Dijkstra. Note that DBPR's low PDR comes at the cost of increased end-to-end delay, its region restriction strategy lacking measures to handle link interruptions in polar regions. During the gradient update interval, there are packets that are forwarded to the polar region, resulting in an increase in PDR. Besides, disabling the transmission region restriction may lead to loop transmissions and TTL

TABLE I
Simulation Parameters

Parameters	Settings
Nubmer of flows	100
On/off model	Pareto distribution
ISL latitude threshold	80°
ISL bandwidth	10 Mbps
Buffer size	30 packets
Packet size	1024B

Authorized licensed use limited to: AJOU UNIVERSITY. Downloaded on November 05,2025 at 11:11:18 UTC from IEEE Xplore. Restrictions apply.

exceed. The LRES, with its hop-by-hop backtracking rerouting mechanism, suffers from cascading congestion problems under heavy load, contributing to a larger PDR. In contrast, ATLB maintains balanced end-to-end delay while enhancing PDR performance.This is attributed to ATLB's global load balancing mechanism, which forwards non-excessive traffic along the default path and redirects excess traffic through relatively uncongested links on the shortest path. Additionally, Fig.4 demonstrates that ATLB and DBPR achieve similar throughput, both surpassing LRES and Dijkstra.

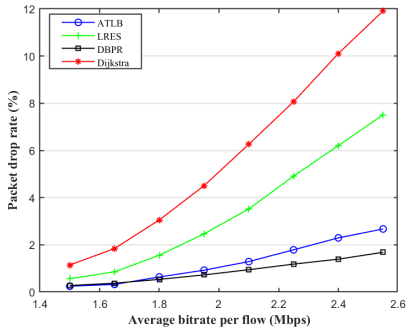


Fig. 3. Packet drop rate versus data generation rate.

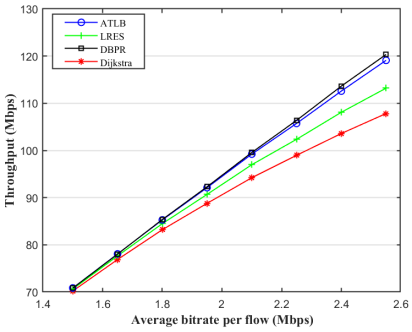


Fig. 4. Throughput versus data generation rate.

As depicted in Fig.5, where ATLB exhibits the best end-to-end delay performance among the compared load balancing routing algorithms. During transient congestion, ATLB redirects excess traffic to alternative paths with only negligible delays. During transient congestion, ATLB redirects excess traffic to alternative paths with only negligible additional delays. During persistent congestion, the global balancing mechanism ensures that delays do not escalate significantly under heavy loads. The LRES incurs larger delays as its rerouting mechanism primarily affects upstream congested links. As for DBPR, the gradient-based forwarding strategy cannot guarantee optimal path distance advantage. Although delay weighting significantly improves DBPR's delay performance, packets may still be forwarded along longer paths than the optimal one when loads are not very heavy, resulting in higher end-to-end delay for DBPR.

V. Conclusion

In this paper, the adaptive timescale load balancing (ATLB) routing algorithm for LEO satellite network is proposed. In

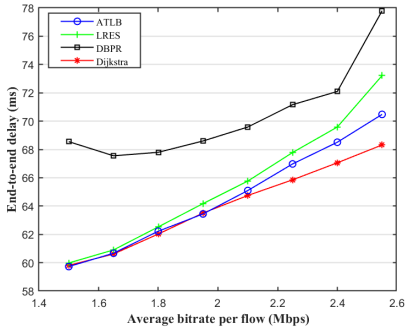


Fig. 5. Average delay versus data generation rate.

ATLB, the link historical traffic is modeled and a congestion classification mechanism is implemented. The ATLB can adaptively adjust the timescale of routing decisions based on the state of link congestion, and complete load balancing at the corresponding timescale. The simulation results show that the ATLB has significant advantages in end-to-end delay, packet drop rate and throughput.

Acknowledgement

This work is supported in part by Shanghai Municipal Science and Technology Major Project under grant 2021SHZDZX0102.

References

[1] A. Gaber, M. A. ElBahaay, A. Maher Mohamed, M. M. Zaki, A. Samir Abdo, and N. AbdelBaki, "5g and satellite network convergence: Survey for opportunities, challenges and enabler technologies," in *2020 2nd Novel Intelligent and Leading Emerging Sciences Conference (NILES)*, 2020, pp. 366–373.

[2] H. Al-Hraishawi, H. Chougrani, S. Kisseleff, E. Lagunas, and S. Chatzinotas, "A survey on nongeostationary satellite systems: The communication perspective," *IEEE Communications Surveys Tutorials*, vol. 25, no. 1, pp. 101–132, 2023.

[3] T. Taleb, D. Mashimo, A. Jamalipour, N. Kato, and Y. Nemoto, "Explicit load balancing technique for ngeo satellite ip networks with on-board processing capabilities," *IEEE/ACM Transactions on Networking*, vol. 17, no. 1, pp. 281–293, 2009.

[4] G. Song, M. Chao, B. Yang, and Y. Zheng, "Tlr: A traffic-light-based intelligent routing strategy for ngeo satellite ip networks," *IEEE Transactions on Wireless Communications*, vol. 13, no. 6, pp. 3380–3393, 2014.

[5] X. Liu, X. Yan, Z. Jiang, C. Li, and Y. Yang, "A low-complexity routing algorithm based on load balancing for leo satellite networks," in *2015 IEEE 82nd Vehicular Technology Conference (VTC2015-Fall)*, 2015, pp. 1–5.

[6] X. Deng, L. Chang, S. Zeng, L. Cai, and J. Pan, "Distance-based back-pressure routing for load-balancing leo satellite networks," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 1, pp. 1240–1253, 2023.

[7] Y. Zhu, L. Qian, L. Ding, F. Yang, C. Zhi, and T. Song, "Software defined routing algorithm in leo satellite networks," in *2017 International Conference on Electrical Engineering and Informatics (ICELTICs)*, 2017, pp. 257–262.

[8] M. M. H. Roth, H. Brandt, and H. Bischl, "Distributed sdn-based load-balanced routing for low earth orbit satellite constellation networks," in *2022 11th Advanced Satellite Multimedia Systems Conference and the 17th Signal Processing for Space Communications Workshop (ASMS/SPSC)*, 2022, pp. 1–8.

[9] Z. Han, C. Xu, G. Zhao, S. Wang, K. Cheng, and S. Yu, "Time-varying topology model for dynamic routing in leo satellite constellation networks," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 3, pp. 3440–3454, 2023.

[10] C. Dong, X. Xu, A. Liu, and X. Liang, "Load balancing routing algorithm based on extended link states in leo constellation network," *China Communications*, vol. 19, no. 2, pp. 247–260, 2022.

Authorized licensed use limited to: AJOU UNIVERSITY. Downloaded on November 05,2025 at 11:11:18 UTC from IEEE Xplore. Restrictions apply.